

docs_chain

Ниједан праћени документ не може изаћи из синхронизације — садржајно хеширан, двосмеран, атомичан.

Сажетак

Docs Chain је универзални, Go-имплементиран двосмерни мотор за пропагацију зависности између докумената и база података. Када се било који члан регистрованог ланца промени — Markdown извор, HTML/PDF/DOCX извоз или SQLite база података — он то открива путем хеша садржаја и пропагира промену кроз све повезане чланове атомично.

Кратак опис

Go мотор који одржава документе и базе података у синхронизацији. Користећи Салса-стил инкременталног прекомпутирања заснованог на хешу садржаја преко DAG-а (Канов тополошки редослед, рано одсецање, двосмерне синхронизационе гране, атомично преименовање + SQLite трансакциони комитови), он регенерише извозе кад год се било који повезан артефакт промени.

Детаљан опис

Docs Chain је оно што направите када сте већ једном превише пута написали исти крхки схелл скрипт „регенериши PDF када се Markdown промени”. Он замењује читав тај жанр ручно писане синхронизационе лепила правим мотором. Моделира документе и базе података пројекта као чланове ланца и, када се било који члан промени, пропагира ту промену кроз све повезане чланове у свим декларисаним смеровима — регенеришући и извозећи атомично тако да ниједан праћени артефакт никада не може изаћи из синхронизације. Дизајн позајмљује своју строгост директно из света инкременталних буилд система, а не из скриптовања: откривање промена заснива се на **хешу садржаја, а не на времену**

модификације, па touch не покреће ништа, а једнобајтна измена покреће тачно оне поновне изградње које треба — без лажних аларма, без пропуштених промена. Формално изражено у једној реченици, то је Салса-стил инкременталног прекомпутирања заснованог на хешу садржаја преко DAG-а, са Кановим тополошким редоследом, раним одсецањем које елиминише непромењене подстабла, декларисаним двосмерним sync гранама са ауторитетом, и атомичним преименовањем плус SQLite трансакционим комитовима, тако да пад система усред пропагације никада не може оставити делимично написан извоз. Дистрибуира се као vasic-digital подмодул и користи као основни део HelixConstitution подмодула, па сваки пројекат који усвоји устав добија Docs Chain одмах и региструје сопствене ланце преко YAML по контексту. Имплементација је искрена у погледу статуса (према уставу §11.4.6): Фазе 1–4 (основни DAG + хеширање, адаптери/трансформације чворова, координатор пропагације са атомичношћу, конфигурационо вођени мулти-контекстни CLI са sync/verify/doctor/graph/watch) су имплементирани и тестиране; Фаза 4б додаје генеричке двосмерне уграђене функције md-to-sqlite/sqlite-to-md (чисти Go, промена на нивоу реда, бајт-стабилан кружни ток) и уграђену функцију colorize-html; Фаза 5 обухвата свеобухватне реал-binary енд-то-енд тестове и означена је као ГРЕЕН. Фазе 6–7 (дистрибуција устава, АТМОСпхере повезивање) остају ПЛАНИРАНЕ и доступне само оператерима. Herald је први прави downstream потрошач, синхронизујући корпус од 66 докумената у више формата који верификује чистоћу.

Зашто смо га направили

Документација, извози и базе података се разилазе чим се одржавају ручно или помоћу крхких скрипти. Docs Chain синхронизацију чини механичком, прецизном на основу хеша садржаја и атомичном, тако да промена било где у ланцу исправно и безбедно ажурира све низводно (и узводно).

Зашто је ово револуционарно

Оно што доноси су ригорозне гаранције исправности, које аутори компајлера и буилд-система сматрају саморазумљивим – графови зависности засновани на хеширању садржаја, минимално понављање израчунавања, атомичне промене – али их усмерава на документацију и базе података, домен који је до сада преживљавао захваљујући црон пословима и добрим намерама. Права двосмерна синхронизација значи да се однос између изворног материјала и његовог извоза одржава у оба

смера, тако да „документација није ажурна” и „извоз се не поклапа са извором” престају да буду понављајући багови и постају стања која систем једноставно неће дозволити.

Шта је иновативно

- Инкрементално поновно израчунавање засновано на хешу садржаја (а не на времену измене) над усмереним ацикличним графом (DAG) са раним прекидом.
- Двосмерна синхронизација са експлицитно дефинисаним ауторитетом (документација ↔ извоз ↔ SQLite).
- Атомична промена имена + SQLite трансакциони комит за безбедно ширење промена чак и у случају пада система.
- Чист Go md-to-sqlite/sqlite-to-md кружни процес са детекцијом одступања на нивоу редова.

Изазови и решења

- **Лажна поновна изградња:** решено детекцијом хеша садржаја уместо временских ознака.
- **Делимичне/корумпиране измене:** решено атомичном променом имена и SQLite трансакцијама.
- **Исправно редослед извршавања са више чланова:** решено Кахновим тополошким сортирањем са раним прекидом.
- **Поштен извештавање о могућностима:** решено означавањем сваке фазе као ИМПЛЕМЕНТИРАНО или ПЛАНИРАНО према §11.4.6.

Технолошки стек (зашто + како)

- **Go** — цео погон (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + Кахново тополошко сортирање** — редослед зависности са раним прекидом.
- **SQLite (чист Go модернц)** — чланови базе података и трансакциони комити.
- **fsnotify** — watch даемон за преношење промена у реалном времену.
- **YAML цонфиг** — регистрација ланаца по контексту.
- **ехес: трансформације** — прикључиви генератори Markdown→HTML/PDF/DOCX.

Поштење у плану развоја: Фазе 6–7 (дистрибуција устава, повезивање са АТМОСфером) су ПЛАНИРАНЕ / доступне само оператерима – нису још испоручене.